

LIVE ONLINE TALK

Do More Dollars Always Buy More AI Intelligence?

Avinash Reddy | PhD Student, UCF

Thursday, Oct 15 • 9pm ET / 6pm PT • Zoom Webinar



Agents already do real work in six domains



Coding

Write, debug and ship code from a prompt

Claude Code, Cursor



Deep research

Search the web, read sources, write cited reports

Deep Research, Perplexity



Computer use

Click, type and navigate apps like a person

Operator, Claude



Customer support

Resolve tickets and answer questions 24/7

Intercom Fin, Sierra



Data analysis

Query data, run analyses and build charts

ChatGPT, Julius



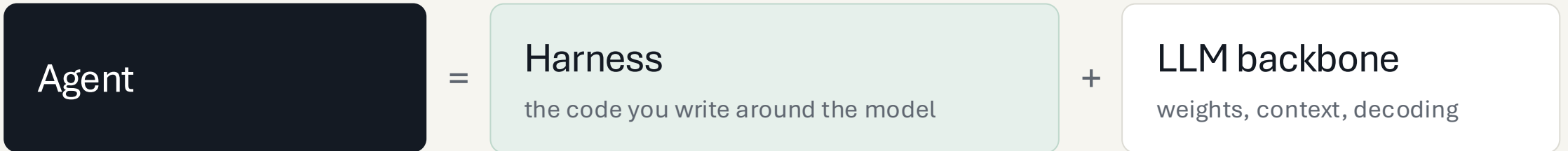
Personal assistant

Triage email, schedule meetings, book travel

Copilot, Gemini

WHAT WE ARE EVALUATING

An agent is a harness plus a backbone



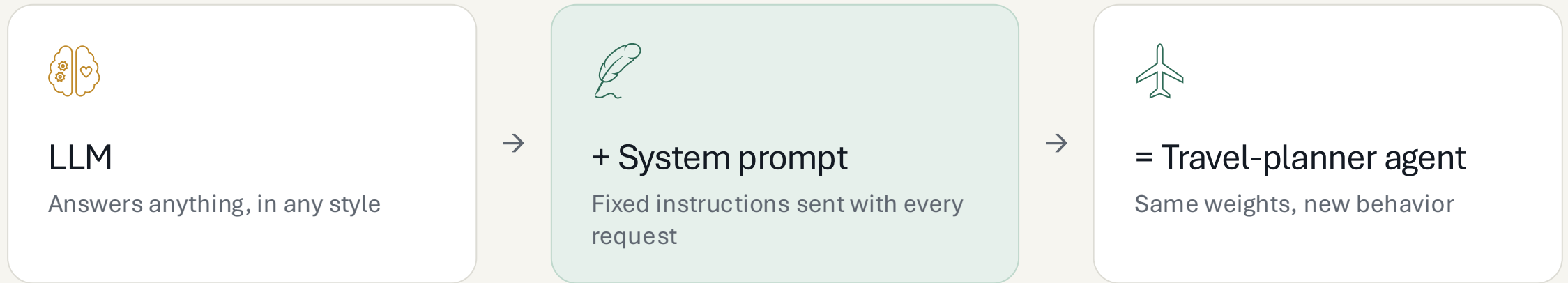
A HARNESS, IN FULL

```
def research(question, budget):
    notes = LLM("break this into sub-questions")
    while budget.left() and not done(notes):
        query = LLM("what should I search for next?")
        notes += search(query)
    return LLM("answer using only these notes")
```

LLM() is the backbone every other line is the harness

One pass through that loop is one turn, and turns are what a budget buys.

The simplest harness is a system prompt



ONE CALL, STEERED BY TEXT

SYSTEM You are a travel planner. Ask for dates and budget first.

USER Plan me a trip to Lisbon.

LLM Happy to help. Which dates, and what is your budget?

No tools and no loop: one LLM call, specialized by its instructions.

Tools let the LLM act, not just answer

USER What will the weather be in Orlando tomorrow?

LLM



Decides to call a tool

`weather("Orlando")`



HARNESS



Runs the tool

→ 84°F, 40% rain



LLM

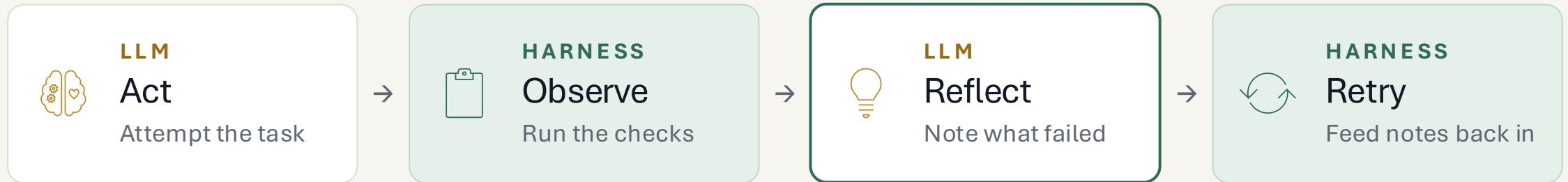


Answers from the result

Warm, with a 40% chance of rain.
Pack an umbrella.

The LLM only writes the call. The harness runs it and hands the result back.

Reflexion hardcodes a reflect step after every turn



REFLEXION, IN FULL

```
def reflexion(task, budget, notes=[]):  
    while budget.left():  
        attempt = LLM(task, notes)           # act  
        result = run_checks(attempt)         # observe  
        if result.passed: return attempt  
        notes += LLM("what went wrong?")    # reflect  
    return attempt
```

The code fixes the order of steps. The LLM only fills each one in.

Same LLM, three harnesses, three kinds of agent



LEVEL 1

System prompt

Specializes behavior with instructions

WHO PICKS THE NEXT STEP

No next step: one call

EXAMPLE

Travel planner



LEVEL 2

+ Tools

Acts through tools; the tools set the agent type

WHO PICKS THE NEXT STEP

The LLM, every turn

EXAMPLE

Coding, browser, deep research, support



LEVEL 3

+ Fixed loop

Runs set steps, with a reflect step after each turn

WHO PICKS THE NEXT STEP

The code

EXAMPLE

Reflexion agent

Each level adds structure around the same LLM, and more LLM calls to pay for.

ONE INFERENCE, ONE BILL

The tokens cost \$0.02. The bill came to \$0.40.

1 PROMPT TO THE LLM ~400 input tokens

```
"Forecast next quarter's sales from sales_2025.csv.  
Write Python and run it in the sandbox."
```



2 LLM OUTPUT ~1,000 output tokens

```
df = pd.read_csv("sales_2025.csv")  
model = fit_forecast(df, trials=200, gpu=True)  
print(model.predict("Q1 2026"), model.score())
```



3 SANDBOX EXECUTION GPU sandbox, ~20 min, billed per second

```
$ sandbox.run(code)  
→ GPU container started, 200 trials executed  
→ 4.2M 0.91
```



4 LLM FINAL ANSWER ~100 output tokens

```
"Q1 2026 sales forecast: $4.2M (fit score 0.91)."
```

INFERENCE BILL

STEPS 1, 2, 4

LLM tokens

\$0.02

STEP 3

Sandbox

\$0.38

Total

\$0.40



Tokens were 5% of the bill

Writing the code was cheap. Running it cost about 19 times as much.

Academic Misconception that tools calls are free

Official Review of Submission22208 by Reviewer r4ZX

Official Review by Reviewer r4ZX 30 Oct 2025, 19:33 (modified: 12 Nov 2025, 05:08) Everyone Revisions

Summary:

This paper shifts the focus of test-time scaling (TTS) from *token-related scaling* to *tool-call-related scaling*, thereby extending the scope of TTS. It introduces a new problem: how to effectively utilize additional computational budgets allocated for tool calls in tool-augmented agents. To address this, the authors design a *cost-effective agent test-time scaling framework* that scales computation for tool calling with budget-awareness. Using *web search* as a representative task, the proposed scaling method is shown to outperform the base model, several existing trained agents, and some token-based TTS methods.

Soundness: 2: fair

Presentation: 2: fair

Contribution: 3: good

Strengths:

1. The paper presents a novel research problem — *test-time scaling under a tool-call budget* — which fills an important gap in current TTS research and is of high research value.
2. The proposed budget-aware planning and balanced budget usage are thoughtful and practically meaningful design considerations.
3. The problem formulation and motivation are clearly written, especially in the early sections of the paper.

Weaknesses:

1. The writing quality fluctuates significantly. The first two sections are well-organized and easy to follow, but the later sections, particularly Section 3, lack critical technical details. While the introduction mentions key challenges such as balancing different budget usage across tools and budget-aware exploration, the method section does not clearly explain how these challenges are addressed. Furthermore, several essential details are missing, which hinder reproducibility — for example, how the agent selects tools, how subsequent planning steps utilize previous tool outputs, and how the agent balances between breadth and depth of exploration. The paper would benefit from additional equations and figures to improve clarity. Finally, the early termination mechanism mentioned in line 398 lacks a corresponding description in the method section, and line 472 contains a capitalization issue.
2. The unified metric introduced in Equation (2) lacks generalization. The definition of *economic cost* does not apply uniformly to both local model usage and local tool functions because the models and tools in local are free.
3. The problem setup, which assumes that each tool call has an explicitly defined budget, is unrealistic. In practice, users typically specify a *total* computational budget, and the allocation across tools should be determined by the proposed method. Moreover, the budget definition should jointly consider both *token budget* and *tool-call budget* to align with the broader definition of test-time scaling.
4. The experiments are insufficient. (1) The main experiment should focus on demonstrating how the proposed method improves the *scaling cost-performance curve*, rather than only comparing with base or parallel models. Figure 2's comparison is therefore inadequate. (2) The choice of TTS baselines is limited. The token-based baselines do not include current state-of-the-art methods, and the paper does not explore tool-call-related scaling methods, such as simple BoN or self-refinement on tool calls. (3) Evaluating on only one type of task is insufficient to demonstrate the method's generalization. (4) The ablation studies in Section 5.3 are not well-designed. The ablation of the planning and budget-tracker modules should be conducted within the proposed framework itself, not on another existing method.

WHERE THE DOLLARS BURN

Tool calls cost dollars too, not just LLM tokens

THE USUAL NOTION ~~cost = LLM tokens~~

Cost of a turn

what the budget pays

=

LLM

Tokens

input and output, per call



+

HARNESS

Tool calls

billed per request by the API



GOOGLE CUSTOM SEARCH API

\$5 per 1,000 queries

`search(query)` → 1 query



SERPAPI

\$10 per 1,000 searches

`search(query)` → 1 search



Every `search()` in the loop is billed, on top of the tokens the LLM spends deciding to make it.

Source: Google Custom Search JSON API pricing (beyond 100 free queries/day); SerpApi plan, \$150/month for 15,000 searches.

Every tool call has a price

	TOOL CALL	WHAT IT DOES	BILLED BY	TYPICAL PRICE
	<code>search()</code>	Web search, Google Custom Search	per query	\$0.005
	<code>serp()</code>	SerpApi (\$150 per 15,000)	per search	\$0.010
	<code>read_page()</code>	Fetch a web page (\$20 per 5,000)	per page	\$0.004
	<code>run_code()</code>	CPU sandbox	per second	~\$0.10 / hour
	<code>run_code(gpu)</code>	GPU sandbox (A10G)	per second	~\$1.10 / hour
	<code>remember()</code>	In-memory store (managed Redis)	per GB-hour	~\$0.125 / GB-hr

ONE RESEARCH RUN 20 searches + 10 page reads ≈ **\$0.14 in tool fees**, before a single token

Prices are typical public list rates (Google Custom Search, SerpApi, Firecrawl, E2B, Modal, AWS ElastiCache); check current pricing.

ONE TASK, SIX METERS

Tokens and tools use different units. Dollars add them up.

	RESOURCE	USAGE IN ONE TASK	× PRICE	= DOLLARS
	Tokens	40K tokens	× \$6 per 1M	\$0.24
	Search calls	20 calls	× \$0.005 per call	\$0.10
	Page reads	10 pages	× \$0.004 per page	\$0.04
	CPU sandbox	0.5 CPU-hours	× \$0.10 per hour	\$0.05
	GPU sandbox	0.33 GPU-hours	× \$1.10 per hour	\$0.36
	Memory store	2 GB-hours	× \$0.125 per GB-hr	\$0.25

ONE BILL 6 units, 1 currency

\$1.04

Tokens are \$0.24 of the \$1.04. The other 77% is billed outside the LLM.

Illustrative usage; prices from the previous slide, tokens at a blended ~\$6 per 1M input+output.

TEST-TIME SCALING, TURN BY TURN

Test time scaling

USER Which city hosted the 1992 Olympics?

LLM Atlanta. **X wrong**

TURN 1

HARNESS Can you check once more and answer?

LLM Atlanta was 1996. 1992 was Barcelona. **✓**

TURN 2

HARNESS `search("1992 Olympics")`

LLM Confirmed by the source: Barcelona. **✓ verified**

TURN 3

RUNNING BILL

Turn 1
1 LLM call **\$0.002**

Turn 2
+1 LLM call **\$0.004**

Turn 3
+1 search, +1 call **\$0.011**

The model never changed. Three turns made the answer correct and verified, at 5.5× the cost.

Illustrative trajectory and costs.

Test-time scaling buys a better answer with more inference

THE DEFINITION

Same model, same weights. **Spend more at inference time** to get a better answer.



1 · CHECK AGAIN

Ask it to verify

"Can you check once more and answer?"

+1 LLM call that re-reads its own answer



2 · RETHINK

Ask it to reason again

"Please rethink."

+ a fresh reasoning pass, more tokens



3 · FORCE MORE STEPS

Force inference or tools

`min_turns = 10`

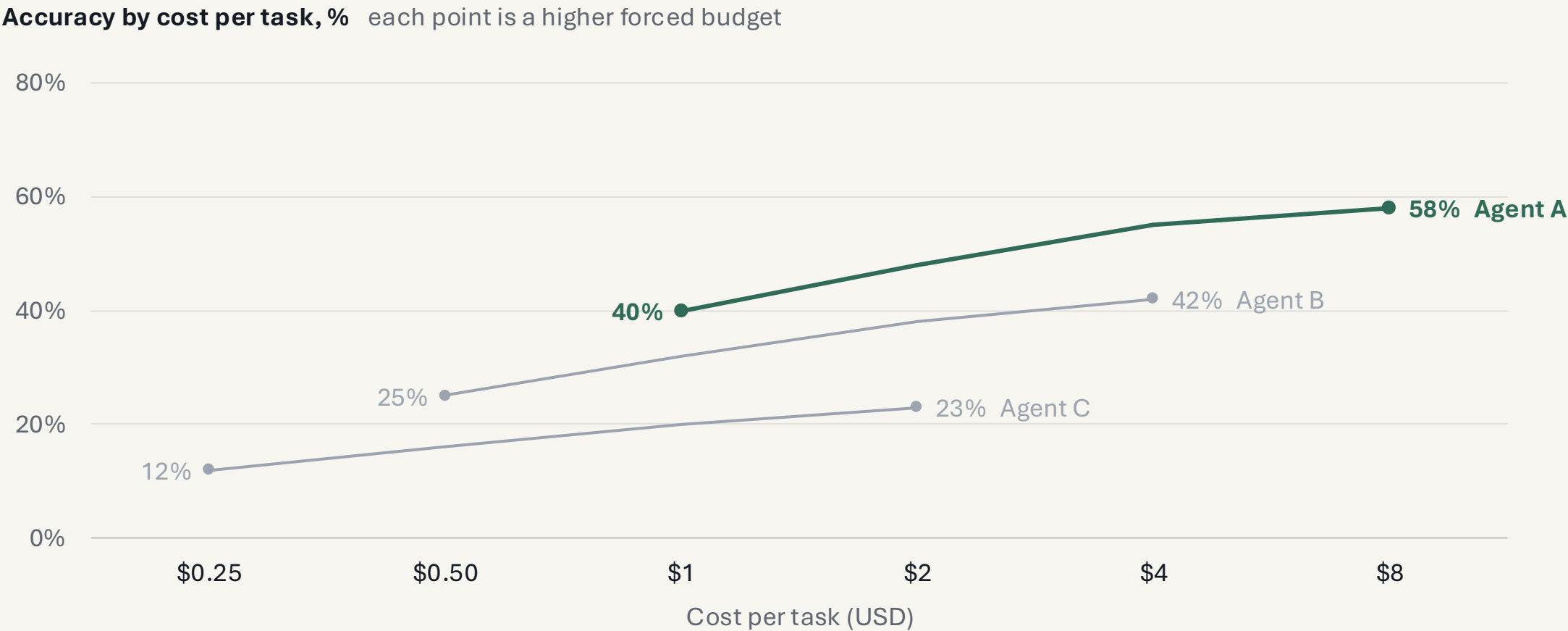
`search() × 10`

+ tokens and tool fees on every turn

Each nudge adds turns, and every turn adds to the bill from the earlier slides.

TEST-TIME SCALING

Every agent scores higher when forced to spend more



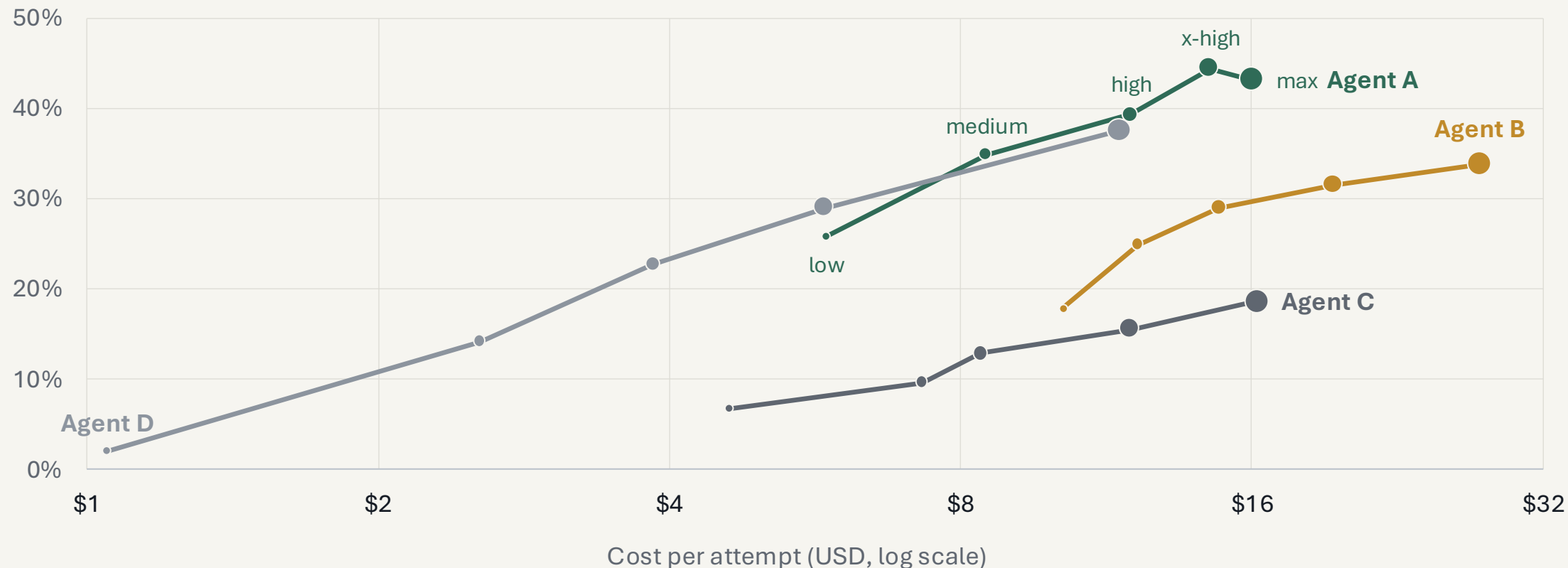
More budget means more turns, more searches and more checks, so accuracy climbs for every agent.

Illustrative curves showing the shape of test-time scaling; not measured results.

THE METRIC

Plot accuracy against total cost, then rank by intelligence per dollar

Agentic coding score by effort level, % larger dot = higher effort setting, low → max



Values approximated from the Frontier-Bench v0.1 effort-level figure; models renamed Agent A–D.

THE PROBLEM

Accuracy and cost alone give four answers to “which agent?”

IF YOU WANT THE HIGHEST SCORE

Agent A

44% at \$14.50 per attempt

IF YOU WANT THE LOWEST COST

Agent D

\$1.05 per attempt, but only 2%

IF YOUR BUDGET IS ~\$12

Agent A or D?

39% vs 38%: a near tie

IF YOU PAY THE MOST

Agent B

\$27.50 for 34%, below A at \$8.50

Two axes show a trade-off, not a winner. **One number has to rank them: intelligence per dollar.**

Values approximated from the Frontier-Bench v0.1 effort-level figure on the previous slide.

THE METRIC

Plot accuracy against total cost, then rank by intelligence per dollar

NOT ENOUGH

~~Accuracy vs token cost~~

Leaves out search, pages, sandboxes and memory



WHAT WE MEASURE

Accuracy vs total cost

Total cost = **tokens** + **every tool call**, in dollars

AXIS 1 · COST

How much did it spend?

Total \$ per task. On its own, it can't tell you if the spend paid off.



AXIS 2 · EFFICIENCY

Intelligence per dollar = **Accuracy** ÷ **Total cost**

Intelligence each dollar bought, on a fixed task set

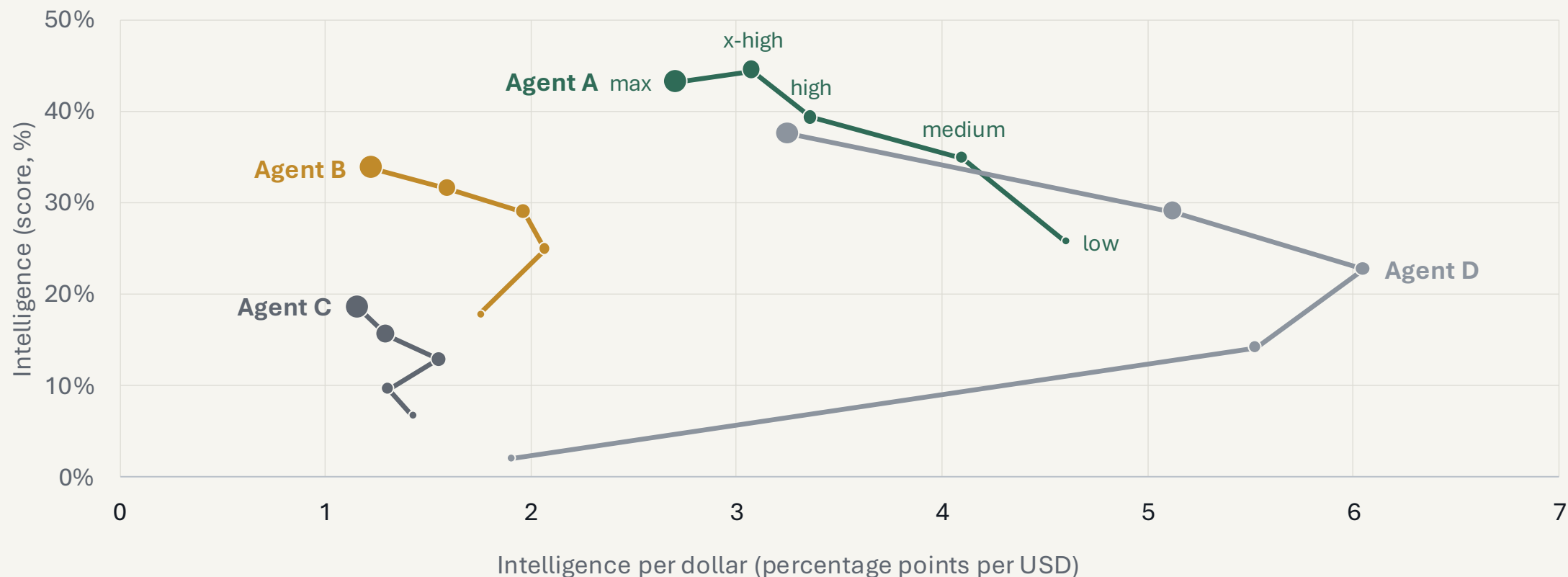
EXAMPLE 100 tasks, 62% correct, \$104 total → **0.6 accuracy points per dollar**

Two agents can score the same accuracy and differ several-fold in dollars per point.

INTELLIGENCE VS INTELLIGENCE PER DOLLAR

Higher effort raises intelligence but lowers intelligence per dollar

Intelligence vs intelligence per dollar, by effort level larger dot = higher effort, low → max



Values approximated from the Frontier-Bench v0.1 score-vs-efficiency figure; models renamed Agent A–D.

TAKEAWAYS

Count every tool call, then pick agents on the frontier

1 Tool calls are not free. Search, pages, sandboxes and memory belong in the cost, not just tokens.

2 The right axes turn a trade-off into a choice

INTELLIGENCE VS COST

Hides the choice

Every curve rises with spend, so each agent looks good at some price.

Highest score, lowest cost and a fixed budget each name a different agent.



INTELLIGENCE VS INTELLIGENCE PER DOLLAR

Shows the frontier

Pick along the frontier by budget:

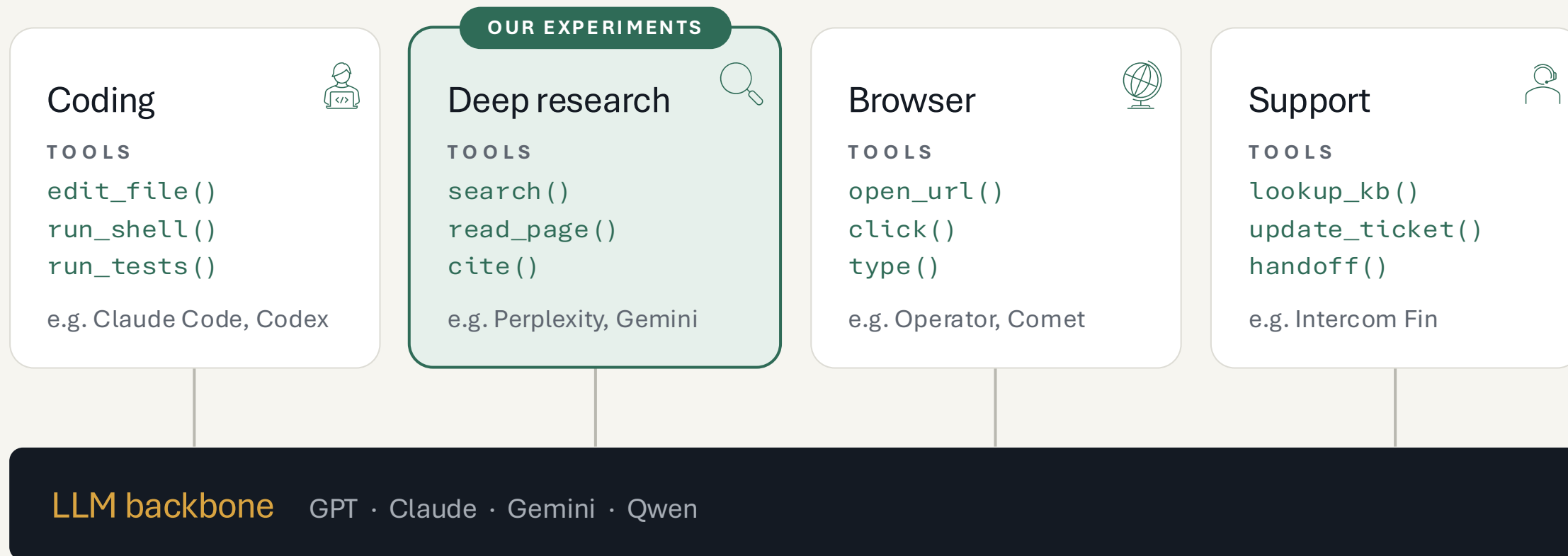
Tight budget Agent D, mid effort: 23–29%

More budget Agent A, medium to x-high: 35–44%

Agents B and C never reach the frontier.

Frontier read from the intelligence-per-dollar chart on the previous slide (approximate values).

Our experiments run on the deep research agent



Every result in this talk comes from the deep research harness.

The deep research agent works through three tools

TOOL CALL	WHAT IT DOES	EXAMPLE OUTPUT
<code>search(txt, top)</code>	Returns the top-k documents for the query, ranked by hybrid BM25 + dense retrieval	<pre>[1] example.com/1 This document... [2] example.com/2 This paper... [3] example.com/3 An overview...</pre>
<code>open(url)</code>	Returns the full parsed page, to read a document end to end	<pre><html><body> ... full page content ... </body></html></pre>
<code>find(url, txt)</code>	Returns a window of text around the first match, for targeted evidence	<pre>... preceding context ... « matched text » ... following context ...</pre>
ONE RUN LLM decides → search candidates → open read → find evidence → repeat → LLM answers		

Every agent uses the same tools over the same deterministic retrieval environment.

SPECIALIZED AGENTS

The scaffold adds specialized LLM agents and memory

LLM AGENTS same backbone, each with its own prompt and job

LLM Thinker

Breaks the question into subgoals

LLM Refiner

Turns subgoals into tool calls

LLM Selector

Picks the relevant documents

LLM Summarizer

Condenses long pages to essentials

LLM Reflexer

Verifies outputs, corrects errors

LLM Conclusion

Writes the final answer from the logs

MEMORY TOOLS where evidence is kept between turns

MEMORY In-context

Keeps evidence inline in the prompt

MEMORY Scratchpad

Persistent store for tool outputs and logs

A SCAFFOLD any mix of these agents and memory, wired to `search · open · find`

Every agent and memory call is metered, so each scaffold has its own cost profile.

THE RATE CARD

Our rate card puts a dollar price on every component

CATEGORY	COMPONENT	INPUT	OUTPUT	WHAT IT IS
Browser	search	\$0.01/call	-	SerpAPI query
	open	\$0.0067/call	-	Scrape and clean up the page
	find	\$0.0067/call	-	In-page lookup
Memory	In-context	\$0.05/M	-	Context always available to the agent
	Scratchpad	\$0.00006/MB	-	Storage for full trajectory logs
Agent scaffold	Refiner	\$0.05/M	\$0.24/M	Turns queries into tool calls
	Selector	\$0.05/M	\$0.24/M	Selects relevant documents
	Summarizer	\$0.05/M	\$0.24/M	Condenses long content
	Conclusion	\$0.05/M	\$0.24/M	Writes the final answer from logs
Backbone	NeMo-N-30B A3B	\$0.05/M	\$0.24/M	Model token pricing

USER-SPECIFIC These are our prices. Swap in your own provider's rates and the same metric applies.

/M = per 1M tokens. Model pricing from the Vercel AI Gateway; search from SerpAPI.

THE TASK

BrowseComp-Plus: hard questions over one fixed corpus

830 questions

Multi-constraint questions taken from BrowseComp

~100K documents

A fixed, curated corpus instead of the live web

1 short answer

Graded against ground truth, so accuracy is exact

WHAT A QUESTION LOOKS LIKE

“A football club founded between 1900 and 1910 was renamed after a merger in the 1990s. Its stadium was designed by an architect who also built a train station. Name the stadium.”

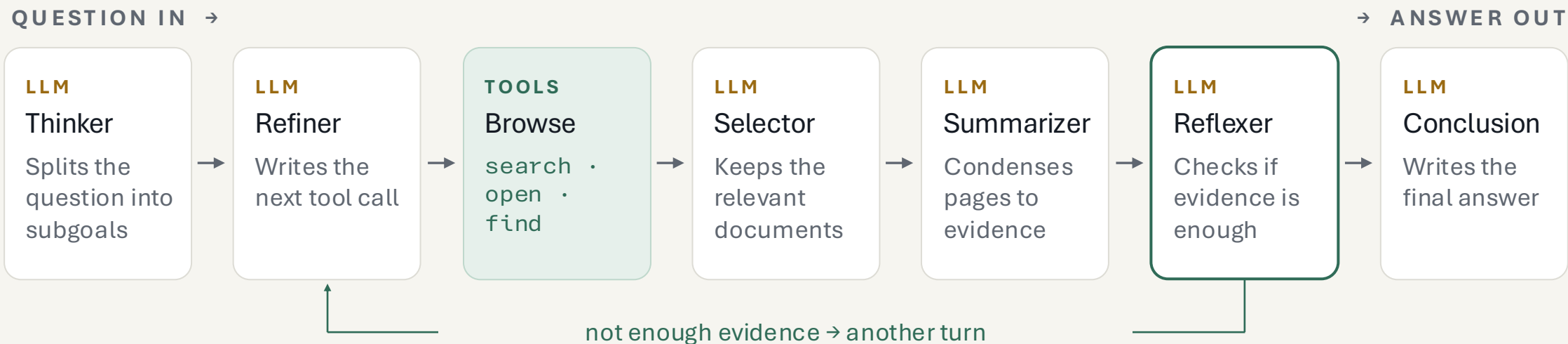
Each clue needs its own search, open and find.

`search()` → `open()` → `find()` × many
→ one short answer, checked for a match

Every agent sees the same pages, so cost differences come only from how many calls it makes.

BrowseComp-Plus (Chen et al., 2025). Example question is illustrative, written in the benchmark's style.

Our deep research agent chains specialists around three tools



SHARED MEMORY **In-context** keeps evidence in the prompt · **Scratchpad** logs every tool output

EVERY BOX IS METERED **LLM boxes** per token, **tools** per call, memory per MB, priced by the rate card

Each extra loop adds a Refiner call, tool calls, a Selector and a Summarizer pass to the bill.

SIX AGENTS WE TESTED

Agents are built from agents: six combinations we tested

AGENT	TOOLS			LLM AGENTS				MEMORY	
	search	open	find	Reflexer	Summarizer	Thinker	Conclusion	In-context	Scratchpad
Search-R1 2 parts	●	—	—	—	—	—	●	—	—
Search-O1 3 parts	●	●	●	—	—	—	—	—	—
ReAct-DR 4 parts	●	●	●	—	—	●	—	—	—
AResearcher 4 parts	●	●	●	—	●	—	—	—	—
IterResearch 7 parts	●	●	●	—	●	—	●	●	●
Reflexion-DR 7 parts	●	●	—	●	●	—	●	●	●

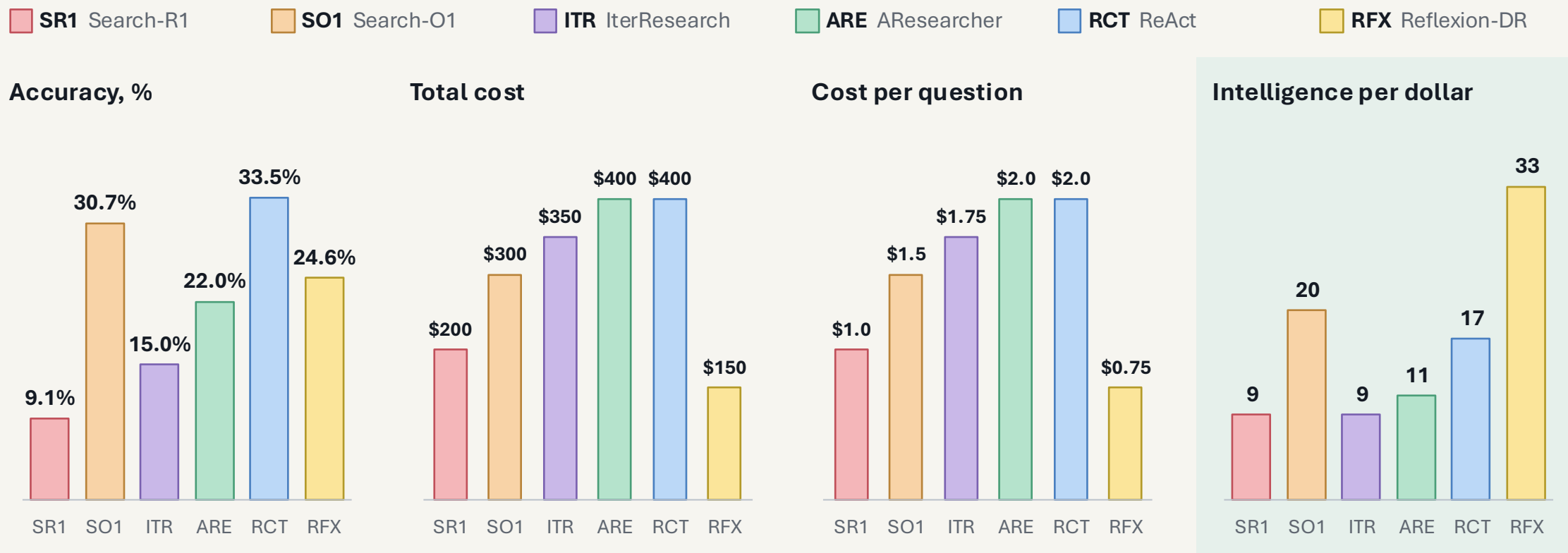
COMPOSITION

Each agent is the backbone plus a set of smaller agents and memory. **More parts, more metered calls.**

Dot = component used. All six run on the same backbone, tools and rate card.

RESULTS · ILLUSTRATIVE

ReAct scores highest, but Reflexion-DR buys the most per dollar

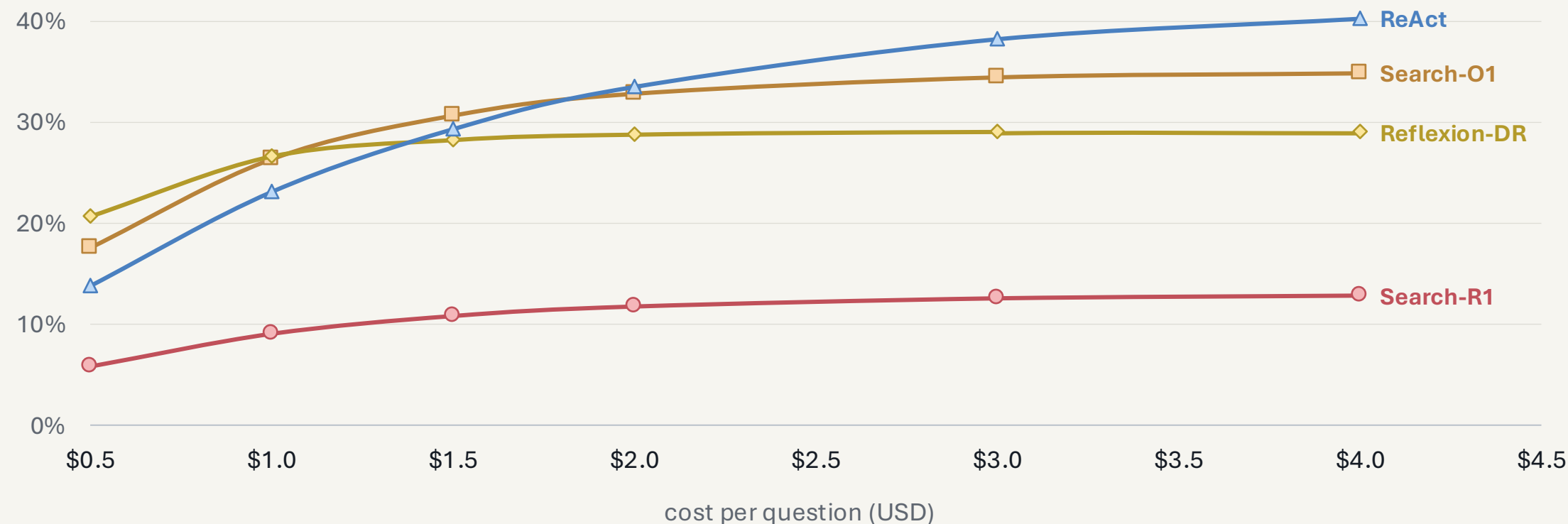


ReAct leads on accuracy, yet Reflexion-DR buys twice its intelligence per dollar.

Illustrative values, matching the scaling and frontier slides; not measured results. Intelligence per dollar = accuracy ÷ cost per question.

Every agent gains with budget, then flattens

Accuracy by budget per question, % each point is a higher forced budget

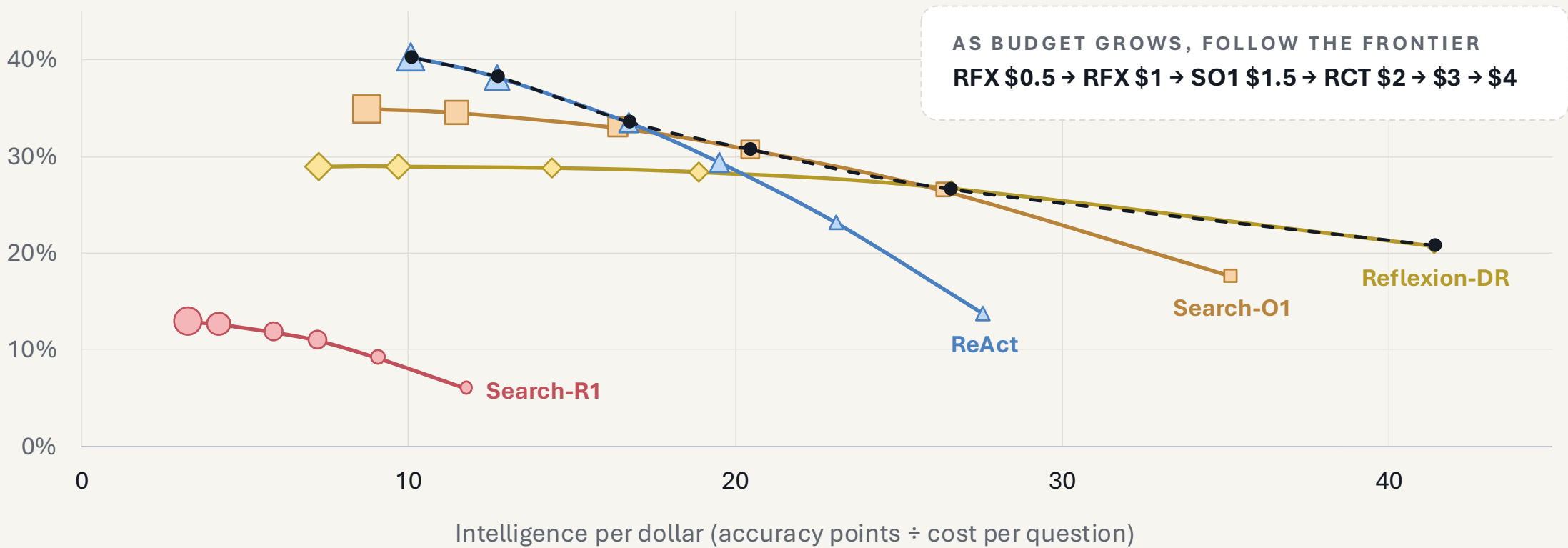


Reflexion-DR leads at low budget, Search-O1 at \$1.5, ReAct from \$2. Past \$3, gains shrink to 1–2 points.

Illustrative values, matching the frontier slide; not measured results.

Walking the frontier picks both the agent and the budget

Intelligence vs intelligence per dollar, by budget dashed line = frontier, larger marker = higher budget



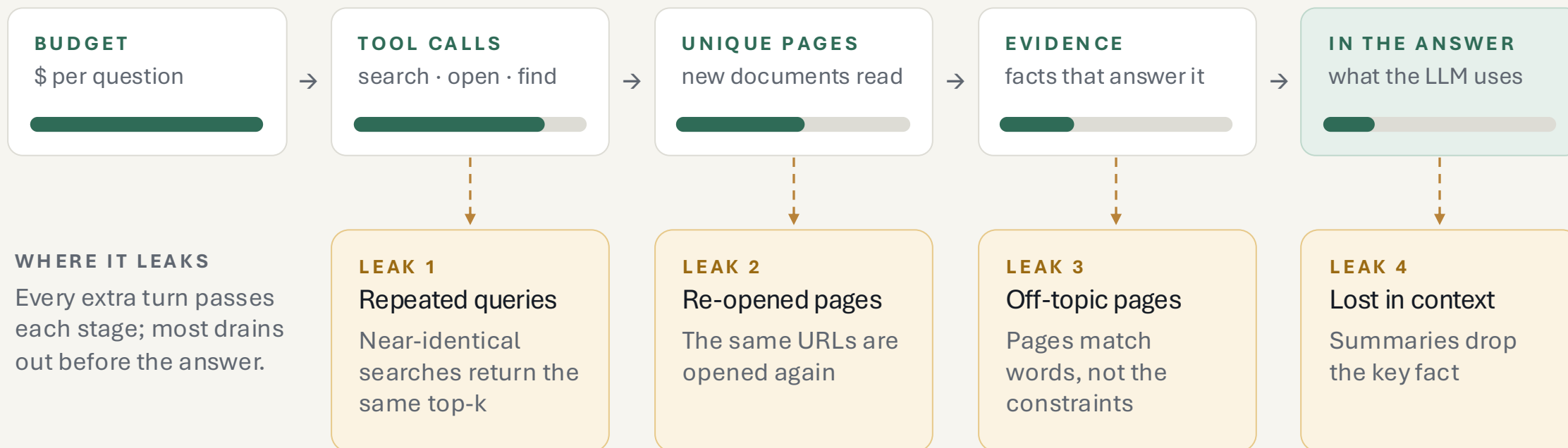
PART 3 • DIAGNOSIS

Why does scaling saturate, and where does it leak?

Following a dollar from budget to answer

WHY SCALING SATURATES

Extra budget leaks into repeat calls, not new evidence



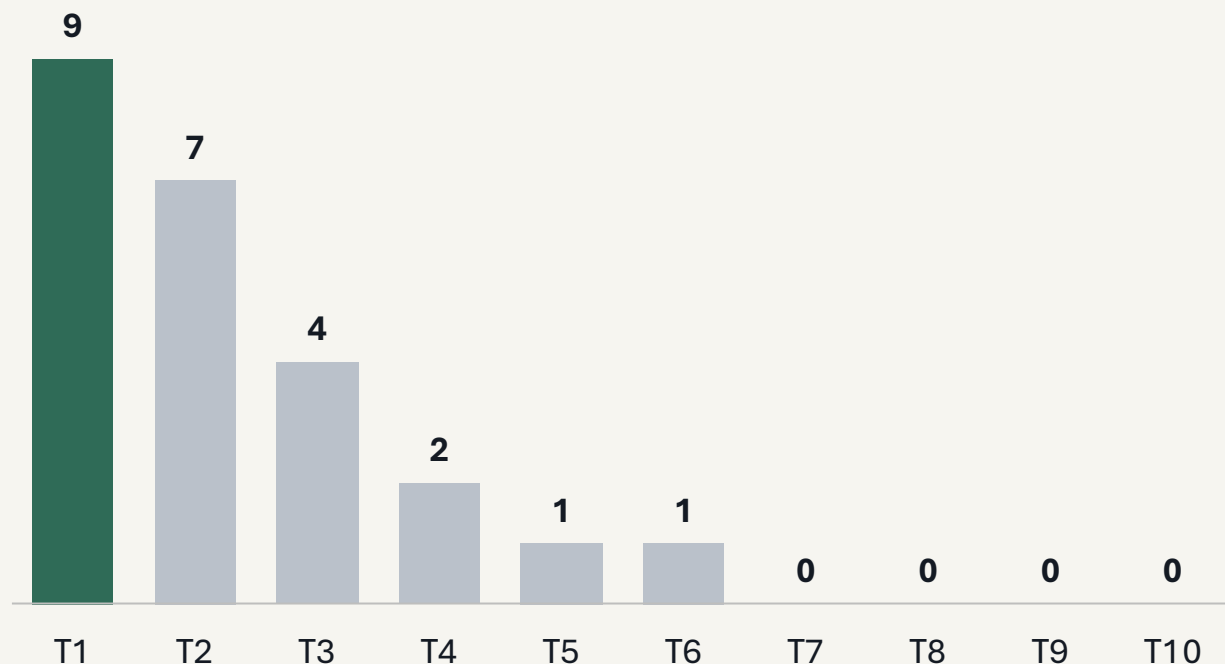
THE PLATEAU Past the plateau, more dollars buy more calls, **not more new evidence.**

A fifth leak: reflection loops re-check the same evidence and rarely change the answer.

Bars are illustrative; leak points are hypotheses to check against trajectory logs.

After a few turns, new searches stop finding new documents

New documents found per turn one question, illustrative



THE AGENT'S SEARCHES

t1 stadium architect station **NEW**
 t2 club renamed merger 1990s **NEW**
 t3 stadium architect railway **3 seen**
 t4 architect station stadium **all seen**
 t5 stadium designed architect **all seen**
 t6 club merger renamed stadium **all seen**

Reworded queries hit the same top-k.

Budget keeps buying searches, but the corpus it reaches stops growing, **so accuracy flattens.**

Illustrative trajectory; measure as unique documents retrieved per turn in the logs.

Pick the hardware setup to fit the application

1 Single GPU

One worker does prefill and decode

2 Multi-GPU, many workers

Each worker serves its own requests

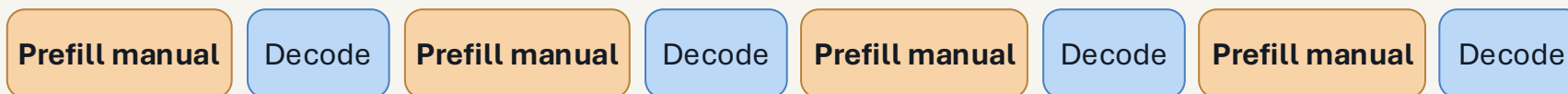
3 Split prefill and decode

Prefill GPUs feed decode GPUs a shared KV cache

EXAMPLE Q&A over a long product manual: every turn re-sends the same huge context

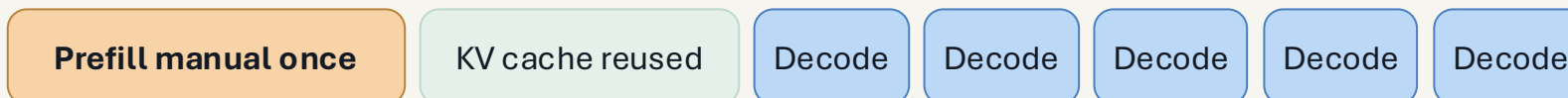
Many workers

context re-processed each turn



Split prefill / decode

context processed once



■ prefill (expensive, scales with context) ■ decode (cheap per turn)

THE LEAK

With many workers, test-time scaling pays to re-read the manual every turn. **Split prefill/decode pays once.**

Schematic, not to scale. Block widths show relative compute per turn.

Agents trained on cheap tools keep calling when prices rise

TRAINING

Tools were cheap

search() price \$0.001

searches per question 40

search bill \$0.04

→
price ×10

DEPLOYMENT

Tools got costly

search() price \$0.01

searches per question 40

search bill \$0.40

A HUMAN

Sees the price and searches less

Picks fewer, sharper queries and stops once the evidence is enough.

THE LEAK The policy never sees the rate card, so a 10× price rise becomes a 10× bill. **Put cost in the loop.**

Tool prices move with the market; an agent's habits are fixed at training time.

Illustrative prices and call counts.

SUMMARY

More dollars buy more intelligence only up to a point

1

No free lunch

Tools are not free.

Search, page reads, sandboxes and memory belong in the bill, not just tokens.

Count every call

2

Intelligence per dollar

Accuracy ÷ total cost.

Walk the frontier to pick the agent and the budget.

Rank on the frontier

3

Diagnose the plateau

Find where extra budget leaks.

Stale queries, re-read context and cost-blind tool use turn dollars into calls, not evidence.

Fix the leak, then scale

DO MORE DOLLARS ALWAYS BUY MORE AI INTELLIGENCE? No. Past the plateau they buy more calls.